

- 2. Side-by-side data comparison (SELECT clause)
- 3. Prepare data before joining (JOIN clause)
- 4. Complex filtering (WHERE clause)
- 5. Checking row existence (EXISTS)

Avoid Subqueries When:

- Simple JOINS are sufficient
- Window functions can accomplish same goal
- CTE (Common Table Expression) is cleaner

CTE (Common Table Expressions)

Definition

A **CTE (Common Table Expression)** is a temporary named result set (virtual table) that exists only for the duration of a single query. It is used to simplify and organize complex queries by breaking them into modular, readable sections.

CTE Internal Logic: How SQL Executes CTEs

SQL processes CTEs in a top-down manner before the main query:

- 1. **CTE Query Execution:** The database engine executes the CTE query first.
- 2. **Virtual Table Creation:** It creates an intermediate virtual table.
- 3. **Caching:** The result is often stored in the **Cache Memory** (very fast memory), making retrieval significantly faster than reading from physical disk storage (HDD/SSD).
- 4. **Main Query Execution:** The main query runs, utilizing both physical database tables and the virtual CTE table stored in cache.
- 5. **Cleanup:** Once the query ends, the virtual table is destroyed and cannot be queried again.

CTE vs. Subquery: Key Differences

Feature	Subquery	CTE
Reusability	Only used once (must rewrite to reuse)	Can be referenced multiple times in the main query
Readability	Bottom-to-Top (often nested and messy)	Top-to-Bottom (organized and modular)
Logic	Intermediate result for single use	Virtual table for multiple uses
Maintenance	Harder to debug in complex queries	Easier to maintain (divided into sections)

CTE Types

1. Non-Recursive CTE

Standard CTE used for typical data retrieval.

- **Standalone CTE:** Runs independently and doesn't rely on other CTEs.

- **Nested CTE:** A CTE that references the result of another CTE defined previously in the same query.

2. Recursive CTE

A self-referencing query that repeatedly processes data (loops) until a specific condition is met. Useful for hierarchical data (e.g., employee-manager charts).

1. Standalone CTE - Basic Syntax

Purpose: Define a single virtual table for a one-time use in the main query.

Syntax:

```
WITH CTE_NAME AS (  
    SELECT column1, column2 FROM table_name WHERE condition  
)  
SELECT * FROM CTE_NAME;  
  
--Example Total Sales per Customer  
  
WITH CTE_TOTAL_SALES AS (  
    SELECT  
        CustomerID,  
        SUM(Sales) as Total_Sales  
    FROM Sales.Orders  
    GROUP BY CustomerID  
)  
SELECT  
    c.CustomerID,  
    c.FirstName,  
    cts.Total_Sales  
FROM Sales.Customers AS c  
LEFT JOIN CTE_TOTAL_SALES AS cts ON c.CustomerID = cts.CustomerID  
ORDER BY c.CustomerID;
```

2. Multiple Standalone CTEs

Purpose: Define several independent virtual tables to combine in the main query.

Syntax:

```
WITH CTE_1 AS (...),  
     CTE_2 AS (...)  
SELECT * FROM CTE_1 JOIN CTE_2 ...;
```

Example:

```

Sales and Last Order Date
WITH CTE_TOTAL_SALES AS (
    SELECT CustomerID, SUM(Sales) as TotalSales FROM Sales.Orders GROUP BY
CustomerID
),
CTE_LAST_ORDER AS (
    SELECT CustomerID, MAX(OrderDate) as LastOrder FROM Sales.Orders GROUP BY
CustomerID
)
SELECT
    c.CustomerID,
    cts.TotalSales,
    clo.LastOrder
FROM Sales.Customers AS c
LEFT JOIN CTE_TOTAL_SALES AS cts ON c.CustomerID = cts.CustomerID
LEFT JOIN CTE_LAST_ORDER AS clo ON c.CustomerID = clo.CustomerID;

```

3. Nested CTE (CTE inside another CTE)

Purpose: Build logic in layers where one CTE processes the result of a previous one.

Example:

```

--Rank and Segment Customers
WITH CTE_TOTAL_SALES AS (
    SELECT CustomerID, SUM(Sales) as TotalSales FROM Sales.Orders GROUP BY
CustomerID
),
CTE_RANK_CUSTOMERS AS (
    SELECT
        CustomerID,
        TotalSales,
        DENSE_RANK() OVER (ORDER BY TotalSales DESC) AS CustomerRank
    FROM CTE_TOTAL_SALES -- References first CTE
),
CTE_CUST_SEGMENT AS (
    SELECT
        CustomerID,
        CASE
            WHEN TotalSales > 100 THEN 'HIGH'
            WHEN TotalSales BETWEEN 50 AND 100 THEN 'MED'
            ELSE 'LOW'
        END AS SalesFrequency
    FROM CTE_TOTAL_SALES -- References first CTE
)
SELECT
    crc.CustomerID,
    crc.CustomerRank,
    ccs.SalesFrequency
FROM CTE_RANK_CUSTOMERS AS crc
JOIN CTE_CUST_SEGMENT AS ccs ON crc.CustomerID = ccs.CustomerID;

```

4. Recursive CTE

Purpose: Handle self-referential data structures until a break condition is met.

Structure:

1. **Anchor Query:** Runs once to provide initial data.
2. **UNION ALL:** Connects anchor to the loop.
3. **Recursive Query:** Self-references the CTE and loops until the WHERE condition breaks it.

Example:

```
--Employee Hierarchy
WITH CTE_EMP_Hierarchy AS (
  -- ANCHOR QUERY: Find the top-level manager (ManagerID is NULL)
  SELECT
    EmployeeID, FirstName, ManagerID, 1 AS Level
  FROM Sales.Employees
  WHERE ManagerID IS NULL

  UNION ALL

  -- RECURSIVE QUERY: Join employees to the levels found previously
  SELECT
    e.EmployeeID, e.FirstName, e.ManagerID, Level + 1
  FROM Sales.Employees AS e
  INNER JOIN CTE_EMP_Hierarchy AS ceh ON e.ManagerID = ceh.EmployeeID
)
SELECT * FROM CTE_EMP_Hierarchy
OPTION (MAXRECURSION 30); -- Safety limit for iterations
```

EXECUTION ORDER

1. Anchor query - Ran only once (which will select the top level employee not having any manager)
2. Recursive query - works by searching if a manager id is equal to which employee id
 - a. first it will check the 1st row which is frank having no manager id so the recursive query will not find any match and will skip it.
 - b. then it will move to the next row (which is kevin) having Frank as manager id so it will find a match and add kevin to the intermediate results with level 2
 - c. then it will move to the next row (which is mary) having frank as manager id so it will find a match and add mary to the intermediate results with level 2
 - d. now next iteration will be started (as now there are no more employees having the frank as their manager id) so the next level will be 3
 - e. from the next line onwards it will keep searching for employees having either kevin or mary as their manager id and will keep adding them to the intermediate results with level 3
 - f. (as the sql stops searching for the employees having the ManagerID as (2(kevin), 3(mary))) now the 2nd iteration will end and 3rd iteration will start which will search if there are any employees having the

managerid as (4 Michael), (5 Carol))this will continue until there are no more employees left to process
(ie no more matches found)

CTE Best Practices & Tips

- **Refactor:** If one CTE can do the work of two, combine them to keep the query lean.
- **Limit Count:** Avoid using more than **5 CTEs** in a single query; it can negatively impact readability and performance.
- **Local Scope:** Remember that a CTE virtual table is only available locally within the query where it is defined.
- **Organize:** Use CTEs for **Modularity** (breaking queries into chunks) and **Reusability** (referencing the same logic multiple times).

End of Comprehensive SQL Notes

Total Topics Covered: 15 major categories

Subtopics: 60+ detailed concepts

Examples: 220+ real-world SQL examples

Definitions: Complete explanations for all functions, operators, and CTE structures

This document represents your complete SQL learning journey covering foundational to advanced concepts in SQL Server!